Благодарю Вас за желание ознакомиться/выполнить ТЗ!
Пожалуйста, обратите внимание, что тестовое задание носит характер проверки навыков и не подлежит оплате;
Выполненные задания с использованием ИИ не принимаются;
Спасибо за понимание 🙏

Тестовое задание: Асинхронный сервис процессинга платежей

Описание

Необходимо реализовать микросервис для асинхронной обработки платежей. Сервис принимает запросы на оплату, обрабатывает их через внешний платежный шлюз (эмуляцию) и уведомляет клиента о результате через webhook.

Сущности

Платеж (Payment)

- ID платежа (уникальный идентификатор)
- Сумма (decimal)
- Валюта (RUB, USD, EUR)
- Описание (строка)
- Метаданные (JSON поле для дополнительной информации)
- Статус (pending, succeeded, failed)
- Idempotency key (уникальный ключ для защиты от дублей)
- Webhook URL (для уведомления о результате)
- Даты создания и обработки

Функционал API

1. Создание платежа

POST /api/v1/payments

- Заголовок: `Idempotency-Key` (обязательный)
- Body: сумма, валюта, описание, метаданные, webhook_url
- Ответ: 202 Accepted, payment_id, статус, created_at

2. Получение информации о платеже

GET /api/v1/payments/{payment_id}

- Ответ: детальная информация о платеже

Технические требования

Брокер сообщений

- При создании платежа публикуется событие в очередь `payments.new`
- Один consumer:
  - Получает сообщение из очереди
  - Эмулирует обработку платежа (2-5 сек, 90% успех, 10% ошибка)
  - Обновляет статус в БД
  - Отправляет webhook уведомление на указанный URL
  - Реализует повторные попытки при ошибках отправки

Гарантии доставки

- Outbox pattern для гарантированной публикации событий
- Idempotency key для защиты от дублей
- Dead Letter Queue для сообщений, не обработанных после 3 попыток

Аутентификация

- Статический API ключ в заголовке `X-API-Key` для всех эндпоинтов

Стек технологий

- **FastAPI** + Pydantic v2
- **SQLAlchemy 2.0** (асинхронный режим)
- **PostgreSQL**
- **RabbitMQ** (FastStream)
- **Alembic** (миграции)
- **Docker** + docker-compose

Требования к результату

1. **Модели и миграции**: таблицы payments и outbox
2. **API эндпоинты**: создание и получение платежа
3. **Consumer**: один обработчик, делающий всё
4. **Outbox pattern**: гарантированная доставка событий
5. **Retry**: 3 попытки с экспоненциальной задержкой
6. **Dead Letter Queue**: для окончательно упавших сообщений
7. **Docker**: compose файл с postgres, rabbitmq, api, consumer
8. **Документация**: README с запуском и примерами

Критерии оценки

- Архитектура и чистота кода
- Корректная реализация Outbox pattern
- Работа с RabbitMQ (очереди, обменники, DLQ)
- Идемпотентность
- Обработка ошибок и retry логика
- Работоспособность Docker-окружения